

1. Non-visual Test Cases

Test Case ID	Test Steps	Test Input	Expected Result	Actual Results	Status	Comments
1.1	Instantiate StockItem class	n/a parameters	n/a	invalid method declaration ; return type required	Fail	n/a response indicates that the class has been instantiated with no issues.
1.2	Instantiate StockItem class	n/a parameters	n/a	n/a	Pass	Error stemmed from incorrect class method application, Java methods require specified data type
2.1	Instantiate StockItem class with constructor (String givenCode, int givenQuantity, double givenPrice)	Parameters: ("0001", 100, 1)	0001 100 1.0	error: constructor StockItem in class StockItem cannot be applied to given types required: no arguments found: String,int,int reason: actual and formal argument lists differ in length	Fail	StockItem constructor method error
2.2	Instantiate StockItem class with constructor (String givenCode, int givenQuantity, double givenPrice)	Parameters: ("0001", 100, 1)	2 100 1.0	n/a	Pass	Changed "public void StockItem(String givenCode, int givenQuantity, double givenPrice)" -> "public StockItem(String givenCode, int givenQuantity, double givenPrice)": "void" was removed
3.1	Instantiate StockItem class with constructor params	Parameters: ("0001", 100, 1)	0001 100 1.0	0001 100 100	Fail	incorrectly, givenQuantity is displayed instead of givenPrice
3.2	Instantiate StockItem class with constructor params	Parameters: ("0001", 100, 1)	0001 100 1.0	0001 100 1.0	Pass	getCost() method incorrectly written to this.quantity instead of this.price
4	Test getVAT func	var getVAT = 17.5	17.5	17.5	Pass	

5.1	Test getStockName(); pre-Set method	name = ""	Unknown Stock Name	Unknown Stock Name	Pass	No name is given. "Unknown ..." should be displayed
5.2	Test getStockDescription() pre-Set method	description = ""	Unknown Description	Unknown Description	Pass	No desc. is given. "Unknown ..." should be displayed
6.1	Test getStockName(); post-Set method	name = "Sponge"	Sponge	Sponge	Pass	
6.2.1	Test getStockDescription() post-Set method	description = "Used with washing liquid"	Used with washing liquid	Sponge	Fail	
6.2.2	Test getStockDescription() post-Set method	description = "Used with washing liquid"	Used with washing liquid	Used with washing liquid	Pass	getStockDescription() method "return this.name" -> "return this.description"
7	Test Set methods	sponge.setCode("0005"); sponge.setQuantity(99); Test sponge.getPrice(2);	0005 99 2.0	0001 100 1.0	Fail	
7.1	Test Set methods	sponge.setCode("0005"); sponge.setQuantity(99); Test sponge.getPrice(2);	0005 99 2.0	0005 99 2.0	Pass	Get methods were assigned to variables prior to Set methods, println method continued to print variables that had not been Set, while the sponge object attributes did in fact change.
8.1	Test setQuantity surpassing 100 (with setQuantity)	(instantiated quantity = 100) sponge.setQuantity(101)	100, ERROR	101	Fail	Quantity should not be able to go beyond 100
8.2	Test setQuantity surpassing 100 (with setQuantity)	(instantiated quantity = 100) sponge.setQuantity(101)	100, ERROR	ERROR: Stock must range from 0 to 100 and not beyond. 100	Pass	if/else conditions fixed
8.3	Test setQuantity surpassing 101 (with addStock)	(instantiated quantity = 100) sponge.addStock(1)	100, ERROR	103	Fail	Unexpectedly, 100 went from 103, let alone 101
8.4	Test setQuantity surpassing 101 (with addStock)	(instantiated quantity = 100) sponge.addStock(2)	102 (for issue diagnosis)	106	Fail	It seems the increase is the integer parameter multiplied by 3 for whatever reason
8.5	Test setQuantity surpassing 101 (with addStock)	(instantiated quantity = 100) sponge.addStock(2)	102 (for issue diagnosis)	102	Pass	Within addStock method, the 0-100 range check performs via a selection of if "this.quantity" is within the bounds. During this

						assessment, (which is performed twice) the arithmetic also occurs, meaning the addition occurs thrice (thus the 3x multiplication). This is fixed by having temp "var x = this.quantity" meaning the parameter "2" is added to 100 only once.
8.6	Test setQuantity surpassing 101 (with addStock)	(instantiated quantity = 100) sponge.addStock(1)	100, ERROR	100 ERROR: Stock must range from 0 to 100 and not beyond.	Pass	Selector logic gates wrong way round. Revised: ((x+=num)>=100 (x+=num)<=0)
8.7	Test setQuantity surpassing 0 (with sellStock)	(instantiated quantity = 0) sponge.sellStock(1)	0, ERROR	0 ERROR: Stock must range from 0 to 100 and not beyond.	Pass	
9.1	Test getVATPrice()	price = 1, VAT = 17.5	1 1.175	1.0 17.5	Fail	VAT is used as the flat rate instead of the percentage VAT being added on top
9.2	Test getVATPrice()	price = 1, VAT = 17.6	1.0 1.175	1.0 1.175	Pass	(VAT * this.price) -> (1 + VAT/100) * this.price)
10.1	Test sellStock/addStock at values 0 and 100	quantity = 0, sellStock(1) quantity = 0, addStock(1) quantity = 100, sellStock(1) quantity = 100, addStock(1)	ERROR, 0 1 99 ERROR, 100	ERROR: Stock must range from 0 to 100 and not beyond., 0 1 ERROR: Stock must range from 0 to 100 and not beyond., 100 101	Fail	sellStock/addStock functions at quantity 0, but seemingly not at quantity 100
10.2	Test sellStock/addStock at values 0 and 100	quantity = 0, sellStock(1) quantity = 0, addStock(1) quantity = 100, sellStock(1) quantity = 100, addStock(1)	ERROR 0 1 99 100	test case quant 0 -1 0 test case quant 100 99 100	Fail	sellStock select function modified from having (x+num) to (x-num) sellStock/addStock now functions at quantity 100, however now does not prompt errors when subtracting

						beyond 0
10.3	Test sellStock/addStock at values 0 and 101	quantity = 0, sellStock(1) quantity = 0, addStock(1) quantity = 100, sellStock(1) quantity = 100, addStock(1)	ERROR 0 1 99 100	test case quant 0 ERROR: Stock must range from 0 to 100 and not beyond. 0 1 test case quant 100 ERROR: Stock must range from 0 to 100 and not beyond. 100 101	Fail	sellStock/addStock parameter operator changed from (OR) to & (AND) Quantity 0 testing is now successful whereas quantity 100 still fails
10.4	Test sellStock/addStock at values 0 and 101	quantity = 0, sellStock(1) quantity = 0, addStock(1) quantity = 100, sellStock(1) quantity = 100, addStock(1)	ERROR 0 1 99 100	test case quant 0 ERROR: Stock must range from 0 to 100 and not beyond. 0 1 test case quant 100 99 100	Pass	Within selection paramaters, a calculation was inside parenthesis, then operated on (i.e (x + num) > 5). This caused issues as "(x + num)" was still treated as an independant insertion operation and changed the original value of x. I fixed this by using "int x = this.quantity + num;" instead of just having "int x = this.quantity" meaning the operation happens only once. This also applies to sellStock but with subtraction instead
11	Test toString()	StockItem sponge = new StockItem("0001", 0, 1);	0001 0 1 1.2	0001 0 1.0 1.2	Pass	1 shows as 1.0 as it is a double

2. Evidence of Functional Methods/Class Instantiation

Part 1.2

Task 1) Create a stock item. 10 units. Unknown Item. Priced 99.99. Item Code W101.

Init:

```
public class Assignment {  
    public static void main(String[] args) {  
        StockItem item1 = new StockItem("W101", 10, 99.99);  
        System.out.println("Task 1")  
        item1.display();  
    }  
}
```

Output:

```
nin@nin-HP-EliteBook-x360-1030-G2:~/Documents/Coding/00Assessment$ java Assignment.java  
Task 1  
Stock Type: Unknown Stock Name  
Description: Unknown Stock Description  
Stock Code: W101  
Price Without VAT: 99.99  
Price With VAT: 119.98799999999999  
Total unit in stock: 10
```

Task 2) Increase units by 10.

Init:

```
public class Assignment {  
    public static void main(String[] args) {  
        StockItem item1 = new StockItem("W101", 10, 99.99);  
        System.out.println("Task 2");  
        item1.addStock(10);  
        item1.display();  
    }  
}
```

Output:

```
nin@nin-HP-EliteBook-x360-1030-G2:~/Documents/Coding/00Assessment$ java Assignment.java  
Task 2  
Stock Type: Unknown Stock Name  
Description: Unknown Stock Description  
Stock Code: W101  
Price Without VAT: 99.99  
Price With VAT: 119.98799999999999  
Total unit in stock: 20
```

Task 3) 2 units sold.

Init:

```
public class Assignment {
    public static void main(String[] args) {
        StockItem item1 = new StockItem("W101", 10, 99.99);
        System.out.println("Task 3");
        item1.addStock(10);
        item1.sellStock(2);
        item1.display();
    }
}
```

Output:

```
nin@nin-HP-EliteBook-x360-1030-G2:~/Documents/Coding/00Assessment$ java Assignment.java
Task 3
Stock Type: Unknown Stock Name
Description: Unknown Stock Description
Stock Code: W101
Price Without VAT: 99.99
Price With VAT: 119.98799999999999
Total unit in stock: 18
```

Task 4) Set new price as 100.99 per unit.

Init:

```
public class Assignment {
    public static void main(String[] args) {
        StockItem item1 = new StockItem("W101", 10, 99.99);
        System.out.println("Task 4");
        item1.addStock(10);
        item1.sellStock(2);
        item1.setPrice(100.99);
        item1.display();
    }
}
```

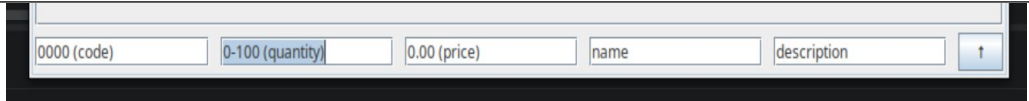
Output:

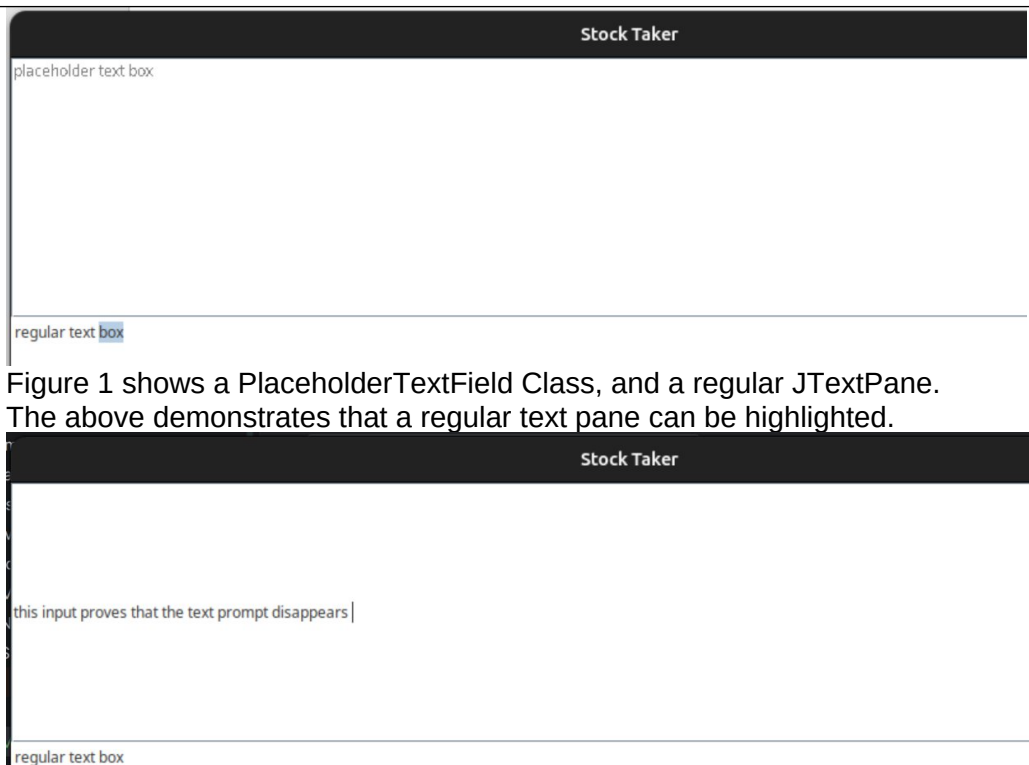
```
nin@nin-HP-EliteBook-x360-1030-G2:~/Documents/Coding/00Assessment$ java Assignment.java
Task 4
Stock Type: Unknown Stock Name
Description: Unknown Stock Description
Stock Code: W101
Price Without VAT: 100.99
Price With VAT: 121.18799999999999
Total unit in stock: 18
```

3. Evidence of Visual Elements

Test 1) Placeholder test

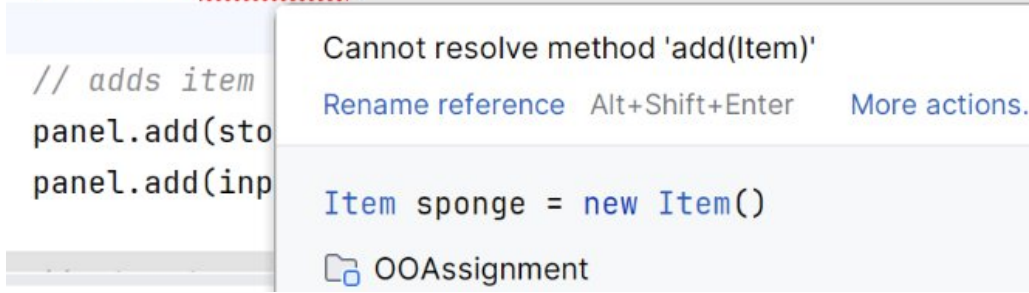
Pass criteria: Text is shown as a hint when input box is empty, and suggestion disappears when input is recieved

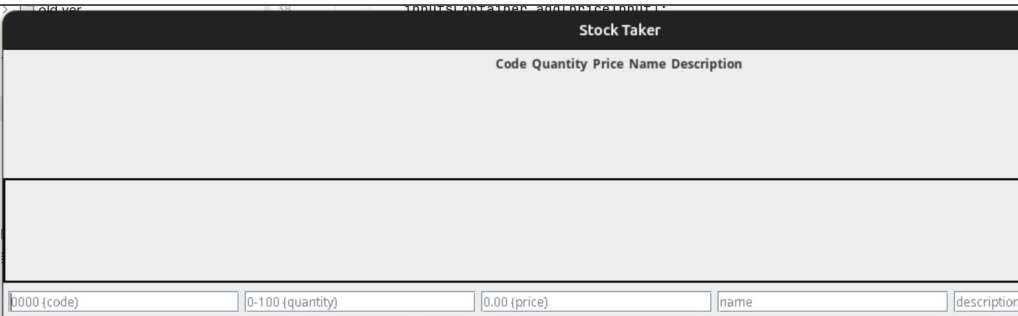
1.1	
Evidence	
Status	FAIL
Comments	Highlight box to see if text disappears. Instead, text becomes highlighted and can be edited by user.

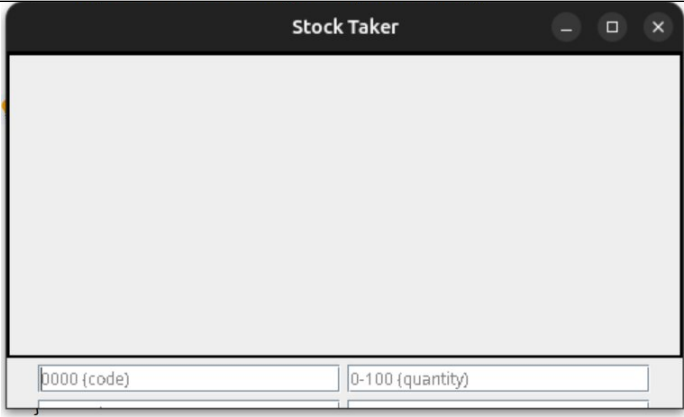
1.2	
Evidence	 Figure 1 shows a PlaceholderTextField Class, and a regular JTextPane. The above demonstrates that a regular text pane can be highlighted. Figure 2 demonstrates that the placeholder text disappears, and can't be highlighted.
Status	PASS
Comments	Text placeholder exists when input box is empty. Disappears when input is given. Text is also given a lighter colour that suggests to the user that input can be recieved.

Test 2) Item example appears in ItemContainer

Pass criteria: Create a 'StockItem' class example which shows inside the 'stockContainer' JPanel, displaying code, quantity, price, name, description.

2.1	
Evidence	<pre>java: no suitable method found for add(Item) method java.awt.Component.add(java.awt.PopupMenu) is not applicable (argument mismatch; Item cannot be converted to java.awt.PopupMenu) method java.awt.Container.add(java.awt.Component) is not applicable (argument mismatch; Item cannot be converted to java.awt.Component)</pre>
Status	FAIL
Comments	Item sponge = new Item(); panel.add(sponge);  Figure 1 suggests that the Class "Item" (a placeholder label) cannot be added to a JPanel. <pre>public class Item { 2 usages JPanel itemContainer = new JPanel(); 6 usages JLabel codeLabel = new JLabel(text: "Code"); 1 usage JLabel quantityLabel = new JLabel(text: "Quantity");</pre> I realised that the itemContainer instance of JPanel within the 'Item' class is used as the main panel for all following gui elements. I shifted the above hierarchy to make the class 'Item' itself an extension of JPanel (as seen below), allowing it access to all of its methods and attributes, which should solve the issue. <pre>public class Item extends JPanel { 2 usages</pre>

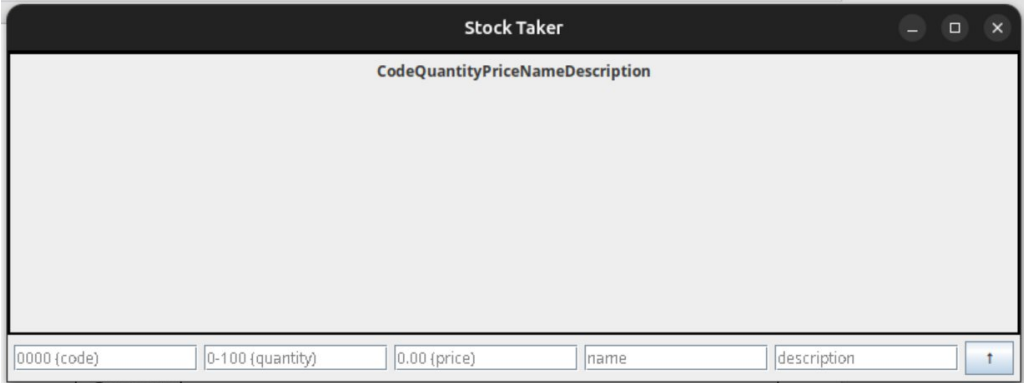
2.2	
Evidence	
Status	FAIL
Comments	The instance of the class now shows visibly within the window. While one criteria has been met, the class instance appears outside the 'stockContainer'.

2.3	
Evidence	
Status	FAIL
Comments	I changed: <code>panel.add(sponge);</code> To: <code>stockContainer.add(sponge);</code> But instance of the class no longer appears in window at all.

2.4	
Evidence	 (no change)
Status	FAIL
Comments	I then attempted: <code>stockContainer.add(new JLabel(text: "test"));</code> The outcome was unchanged, revealing that the issue may not lie within my made class 'PlaceholderTextField', as an ordinary 'JLabel' also does not appear.

2.5	
Evidence	
Status	FAIL

Comments	To reveal where the problem lies, I tested various other routes: <pre>inputsContainer.add(sponge); stockContainer.add(new JLabel(text: "test"));</pre> While text is added to the inputsContainer, such is not the case for stockContainer. This suggests the issue lies with adding a JLabel or PlaceholderTextField (an extension of a JTextField) to the stockContainer (which is a JScrollPane).
----------	--

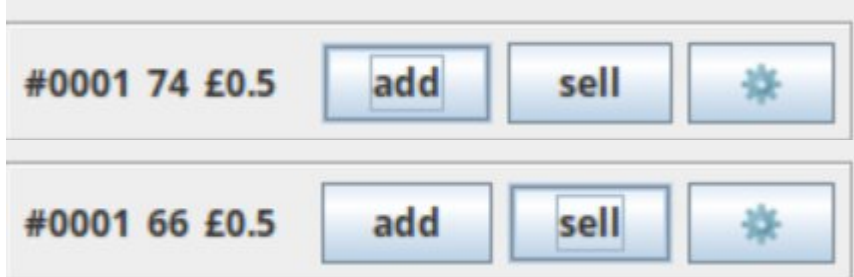
2.6	
Evidence	
Status	PASS
Comments	After researching the JScrollPane Class, I realised that instantiation of the class had an optional parameter of an element to display. I declared a new JPanel 'stockPane' where content is appended to, instead of directly to the JScrollPane 'stockContainer', the change can be seen below: <pre>JPanel stockPane = new JPanel(); JScrollPane stockContainer = new JScrollPane(stockPane);</pre>

Test 3) Functions 'addStock()' and 'sellStock()' affect each stock item.

Success criteria: when add and sell stock buttons are clicked, quantity of stock item is shown visually.

3.1	
Evidence	 Quantity (second column) is shown to be unchanged at 70.
Status	FAIL
Comments	No change is shown. Using System.out.println(quantity) before and after functions, the stock is shown to remain 70 before and after. The 70 output above is as a result of checking quantity values within the event listener.

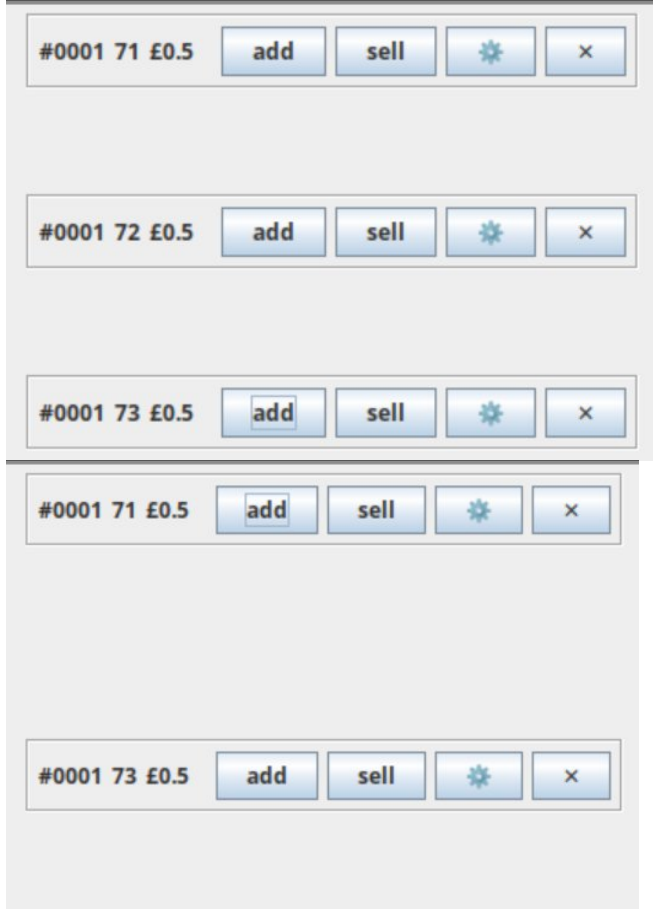
	The 67/68 display outputs are from checking quantity values after clicking add stock and sell stock buttons several times. This suggests the methods are working appropriately, but the issue lies instead within the state of variables within the constructor method, or within the event listeners, potentially a scope issue. <pre>quantity = 1; // added increase decrease button fu increaseButton.addActionListener(new public void actionPerformed(Acti quantity = addStock(num: 1);</pre> The image above proves that the issue is a scoping issue with the event listener.
--	--

3.2	
Evidence	
Status	PASS
Comments	Add and sell stock buttons now function. <pre>increaseButton.addActionListener(new ActionListener() { public void actionPerformed(ActionEvent e) { addStock(num: 1, quantityLabel); }; }); decreaseButton.addActionListener(new ActionListener() { public void actionPerformed(ActionEvent e) { sellStock(num: 1, quantityLabel); }; }); public void addStock(int num, JLabel label) { //(x+=nu int x = this.quantity + nu if (x>=0 && x<=100) { this.quantity += num; label.setText(String.valueOf(this.quantity));</pre> Scope issue was solved by including a JLabel as a parameter in the methods, then updating the label within the add/sell stock methods.

Test 4) Delete stock items

Success criteria: A delete button which removes stock item instance from the scroll pane

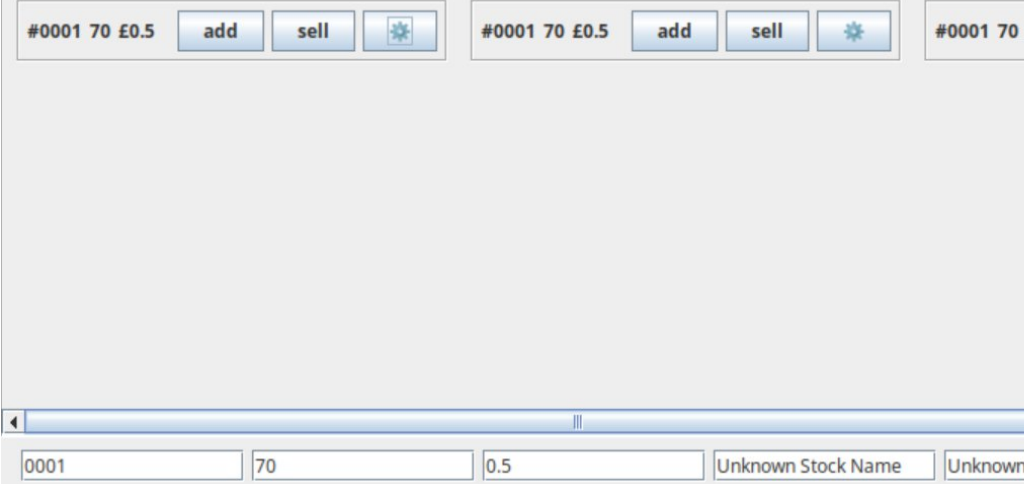
4.1

Evidence	
Status	PASS
Comments	The X button removes the instance from the scrollPane

Task 5) Allow stock editing of items.

Success criteria: When edit button is clicked, pass stock item info into the editor panel at the bottom. Allow new info to be saved.

5.1	
Evidence	

	
Status	PASS
Comments	When the edit button is pressed, the function passToInput() from the class StockItem receives PlaceholderTextField objects from the InputsContainer in the Menu class. When submit button is pressed again, the item re-enters the stockPane.